

Desafio Técnico

3 Projetos em Python — BSC + Backend + Grafana

Este é um desafio prático dividido em **3 projetos independentes**, que somados avaliam sua capacidade de aprender rápido, construir um sistema completo e documentar bem. Prazo total: **7 dias**.

Como funciona

- **Projeto 1** é curto — serve para eu ver se você aprende rápido. Faça nos primeiros 1-2 dias;
- **Projeto 2** é o coração do desafio: pipeline de coleta + banco + API backend;
- **Projeto 3** é a camada de visualização em Grafana, consumindo o banco do Projeto 2;
- **Opcionais** são diferenciais. Faça **depois** de entregar os 3 projetos principais.

Regras importantes

- Tudo em **Python 3.11+**, código público no GitHub, um repositório por projeto (ou um repo com 3 pastas — sua escolha);
- Pode usar Google, documentação, ChatGPT/Claude para tirar dúvidas. **Não pode** entregar código que você não saiba explicar linha por linha;
- Na call final você vai compartilhar tela, rodar tudo ao vivo e responder perguntas técnicas;
- Commits ao longo da semana, não tudo no último dia. Isso é avaliado.

Importante: eu vou te dar os recursos técnicos que você precisa (RPC, endereços, tokens). Tudo mais — como conectar, como converter saldos, como tratar erros, como modelar o banco — você pesquisa sozinho. Isso faz parte do teste.

Recursos BSC (usados nos 3 projetos)

Abaixo estão os endereços e endpoints que você vai usar. Todos são públicos, não precisa de chave de API.

RPC públicos da BSC (escolha um, tenha outro de backup)

```
https://bsc-dataseed.binance.org/  
https://bsc-dataseed1.defibit.io/  
https://bsc-dataseed1.ninicoin.io/  
https://bsc.publicnode.com/
```

Tokens padrão para consulta (BEP-20)

Símbolo	Contrato na BSC
USDT	0x55d398326f99059fF775485246999027B3197955
BUSD	0xe9e7CEA3DedcA5984780BafC599bD69ADd087D56
CAKE	0x0E09FaBB73Bd3Ade0a17ECC321fD13a19e81cE82
WBNB	0xbb4CdB9CBd36B01bD1cBaEBF2De08d9173bc095c

Wallets públicas da Binance para monitorar

Essas carteiras são conhecidas, públicas, e se movimentam o tempo todo. São perfeitas para o desafio.

Nome	Endereço
Binance Hot Wallet 8	0xF977814e90dA44bFA03b6295A0616a897441aceC
Binance 14	0x28C6c06298d514Db089934071355E5743bf21d60
Binance 7	0x3f5CE5FBFe3E9af3971dD833D26bA9b5C936f0bE
Binance Peg Tokens	0x8894E0a0c962CB723c1976a4421c95949bE2D4E3

Biblioteca obrigatória para BSC

Use **web3.py** (pacote web3 no pip). É o padrão da indústria para interagir com redes EVM-compatible. A BSC é uma fork do Ethereum, então funciona igualzinho.

PROJETO 1 | Script de Consulta BSC

Objetivo

Um único script Python que conecta na BSC e mostra o saldo de BNB nativo e o saldo de um token BEP-20 de uma das wallets públicas da Binance acima. Esse é seu **esquenta** — quero ver se você consegue aprender web3.py do zero em 1-2 dias.

O que entregar

- Um arquivo `main.py` que, ao rodar com `python main.py`, imprime na tela:
 - — O bloco atual da BSC;
 - — O saldo de **BNB nativo** de uma das wallets Binance;
 - — O saldo de **USDT** (ou outro token da lista) da mesma wallet;
 - — Tudo em valores legíveis para humano (não em wei);
- Arquivo `requirements.txt` com as dependências;
- `README.md` curto explicando como rodar;
- Endereço da wallet e do token devem ser configuráveis (não pode estar hardcoded).

O que você vai precisar estudar

- Instalar e usar a biblioteca `web3.py`;
- Conectar a um nó RPC via `HTTPProvider`;
- Diferença entre **saldo nativo** (BNB) e **saldo de token** (BEP-20);
- Como chamar funções de um contrato inteligente pelo Python.

Critério de aprovação deste projeto: script roda de primeira no meu computador depois de `pip install -r requirements.txt`, e você consegue me explicar por que o saldo bruto que o RPC retorna não é o número que aparece na tela.

PROJETO 2 | Pipeline + Backend FastAPI

Objetivo

Evoluir o script do Projeto 1 para um sistema de verdade: coleta periódica dos saldos de várias wallets Binance em um token, salva em PostgreSQL, e expõe os dados via uma API FastAPI.

O que entregar

- **Coletor**: função/módulo que, quando executado, consulta o saldo de **USDT** (ou token de sua escolha da lista) em **pelo menos 3 wallets** da Binance e grava no banco com timestamp;
- **Banco PostgreSQL**: com tabela modelada por você para armazenar as coletas (pense no que você precisa: wallet, token, saldo, quando foi coletado);
- **API FastAPI** com no mínimo 3 endpoints:
 - — GET /balances/latest — últimos saldos de todas as wallets monitoradas;
 - — GET /balances/{wallet}/history — histórico de uma wallet específica;
 - — GET /stats — estatísticas (ex: wallet com maior saldo, variação 24h);
- Swagger (/docs) funcionando — o FastAPI faz isso de graça se você usar type hints;
- Código separado em módulos — nada de script monolítico de 400 linhas;
- .env.example com variáveis (RPC, credenciais do banco), mas com .env fora do git;
- README.md com arquitetura, como subir o banco, como rodar o coletor, como subir a API.

O que você vai precisar estudar

- **FastAPI**: path operations, query parameters, response models com Pydantic;
- **PostgreSQL**: instalar localmente (ou rodar via Docker — sua escolha), criar database, conectar via Python;
- **SQLAlchemy** (recomendado) ou psycopg2 puro — você decide;
- **python-dotenv** para carregar configurações do .env;
- Como estruturar um projeto Python com múltiplos módulos.

Critério de aprovação deste projeto: eu rodo o coletor, vejo dados no banco. Rodo a API, abro o Swagger, todos os endpoints respondem com dados reais. Você me explica por que modelou a tabela desse jeito e o que aconteceria se o RPC caísse no meio da coleta.

PROJETO 3 | Dashboard Grafana

Objetivo

Montar um dashboard Grafana que lê **direto do PostgreSQL** do Projeto 2 e mostra, de forma clara, o que está acontecendo com as wallets monitoradas.

O que entregar

- Grafana rodando localmente, com datasource PostgreSQL configurado apontando para o banco do Projeto 2;
- Um dashboard com **no mínimo 4 painéis**:
 - — **Série temporal**: saldo das wallets ao longo do tempo (várias linhas no mesmo gráfico);
 - — **Stat**: saldo total atual somando todas as wallets monitoradas;
 - — **Tabela**: última coleta de cada wallet (endereço, saldo, quando foi coletado);
 - — **Bar chart ou Gauge**: variação percentual das últimas 24h por wallet;
- Queries SQL dos painéis **escritas por você** — importar dashboard pronto da internet é reprovação;
- Exportar o dashboard em JSON e salvar em `grafana/dashboard.json` no repo;
- Print do dashboard funcionando no README.

O que você vai precisar estudar

- Instalar Grafana (download direto ou container — sua escolha);
- Adicionar datasource PostgreSQL no Grafana;
- Tipos de painel: Time series, Stat, Table, Bar chart, Gauge;
- Query editor do Grafana usando SQL (não é SQL puro — tem variáveis especiais de tempo);
- Como exportar/importar dashboards em JSON.

Critério de aprovação deste projeto: eu abro o Grafana, vejo os 4 painéis com dados reais, e você me explica cada query SQL que escreveu. Pergunta favorita: por que o Grafana lê do banco e não da sua API FastAPI?

Opcionais (diferenciais)

Se sobrar tempo **depois** dos 3 projetos entregues e funcionando, implemente um ou os dois. Aqui é com você — sem roteiro, sem dica. O que você aprendeu nos 3 projetos dá base.

Opcional A — Coleta automática agendada

Fazer a coleta rodar sozinha a cada X minutos (ex: 10 em 10 min), sem você precisar executar o script na mão. Deve rodar junto com a API FastAPI ou como processo separado. Se desligar e ligar de novo, continua de onde parou sem perder dados.

Opcional B — Sistema de alerta de variação

Detectar quando uma wallet monitorada teve variação de saldo acima de um limite (ex: saldo mudou mais de 5% desde a última coleta) e registrar esse evento. Deve ter: tabela própria no banco para alertas, endpoint na API GET `/alerts` listando alertas recentes, e um painel no Grafana mostrando quando os alertas dispararam.

Ordem de ataque sugerida (7 dias)

Isto é uma sugestão, não uma prisão. Ajuste ao seu ritmo — mas se gastar mais de 2 dias no Projeto 1, algo está errado e é hora de pedir ajuda.

Quando	O quê	Foco
Dias 1-2	Projeto 1	Aprender web3.py, conseguir ler saldo nativo e de token na BSC.
Dias 3-5	Projeto 2	Modelar banco, criar coletor que salva no Postgres, montar API FastAPI.
Dia 6	Projeto 3	Instalar Grafana, configurar datasource, montar os 4 painéis.
Dia 7	Polimento	README, prints, opcional se der tempo, commit final, preparar para a call.

Onde estudar (tudo grátis)

web3.py

- Documentação oficial: `web3py.readthedocs.io`
- Procurar exemplos de "web3.py balanceOf ERC20" (funciona igual na BSC)

FastAPI

- Documentação oficial: `fastapi.tiangolo.com` — é o melhor tutorial que existe

PostgreSQL + SQLAlchemy

- SQLBolt para aprender SQL básico (interativo)
- SQLAlchemy 2.0 — tutorial oficial, seção "Unified Tutorial"

Grafana

- Documentação oficial do Grafana, seção "Getting Started"
- Para o datasource: pesquisar "Grafana PostgreSQL data source"

BSC / Blockchain

- BscScan (`bscscan.com`) — use para conferir se seus saldos batem
- Documentação BNB Chain: `docs.bnbchain.org`

Dica: sempre que consultar um saldo via web3.py, confira o mesmo valor no BscScan. Se bater, está correto. Se não bater, tem algo no seu código que precisa ser ajustado.

Como será a avaliação

No final da semana agendamos uma call. Não é entrevista — é uma **demo** ao vivo. Você vai:

- Compartilhar tela e rodar cada um dos 3 projetos na sua máquina;
- Abrir o código e me guiar pela arquitetura;
- Abrir o Grafana e explicar cada painel + cada query SQL;
- Responder perguntas específicas sobre decisões técnicas e o que você aprendeu no caminho;
- Contar o que travou e como você resolveu (uso de documentação, Stack Overflow, IA, etc.).

Checklist final antes de me chamar

- Repositório(s) no GitHub, público, com commits ao longo da semana;
- README.md em cada projeto explicando o que é e como rodar;
- requirements.txt em cada projeto;
- .env.example onde houver configuração (sem senhas reais);
- Prints no README do Projeto 3 mostrando o Grafana com dados;
- Testei tudo do zero num diretório limpo — roda de primeira;
- Sei explicar cada arquivo que comitei.

Se tiver dúvida sobre **escopo** ("isso conta como entrega?") me pergunte antes. Se tiver dúvida **técnica**, pesquise primeiro — saber pesquisar faz parte da avaliação. Boa sorte.